DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Design and Verification of a 4-bit Ripple Carry Adder

MINI PROJECT 1 | DIGITAL DESIGN

Prepared for: VLSI Design Project Submission

Institution: VTU, VIAT, Muddenahalli

Student: Shamshad Babajan Ghodesawar

Academic status: ECE, 3rd semester

Report status: Proposed design and verification plan. Numerical simulation results are not claimed until tools are run.

Abstract

This mini project proposes a synthesizable 4-bit ripple carry adder (RCA), verified with a self-checking digital testbench. It demonstrates modular arithmetic design, carry propagation and the trade-off between simplicity and speed.

1. Problem statement

Design a 4-bit combinational circuit that adds two unsigned binary numbers with a carry input, producing a 4-bit sum and carry output. Verify correct operation for all possible input combinations.

2. Objectives

• Implement a reusable 1-bit full-adder module and cascade four stages.

• Create synthesizable Verilog HDL and a self-checking testbench.

• Measure functional correctness and estimate logic depth and area after synthesis.

• Compare an RCA conceptually with a carry look-ahead structure.

3. Literature review

Conventional ripple carry adders are composed of cascaded full adders. They offer straightforward structure and modest hardware complexity, but worst-case carry delay grows with word length. A carry look-ahead adder computes carry signals from generate/propagate relationships, trading extra logic for reduced carry-chain delay.

4. Block diagram

Each stage receives Ai, Bi and carry Ci; it produces Si and carry Ci+1. The final carry C4 extends the unsigned result to five bits.

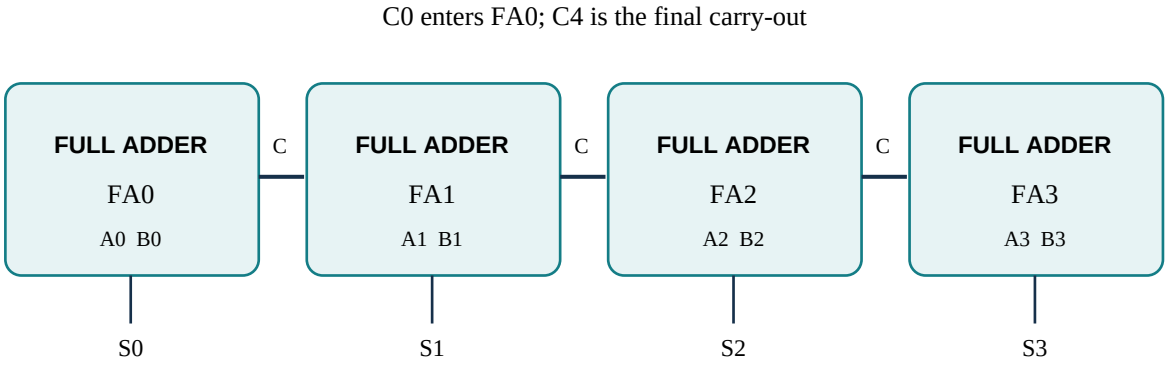

Figure 1. Four-stage ripple carry adder architecture

5. Logic and equations

For each bit i: Si = Ai XOR Bi XOR Ci; Ci+1 = AiBi + Ci(Ai XOR Bi). Four stages are connected in sequence, with C0 = Cin.

6. RTL implementation

```
module full_adder(input a,b,cin, output sum,cout);
assign sum = a ^ b ^ cin;
assign cout = (a & b) | (cin & (a ^ b));
endmodule

module rca4(input [3:0] a,b, input cin,
output [3:0] sum, output cout);
wire [4:0] c;
assign c[0] = cin;
genvar i;
generate for(i=0;i<4;i=i+1) begin: FA
full_adder u(a[i],b[i],c[i],sum[i],c[i+1]);
end endgenerate
assign cout = c[4];
endmodule
```

7. Verification plan

Enumerate 16 x 16 x 2 = 512 combinations of A, B and Cin. For each, compute expected = A + B + Cin using a five-bit reference value and compare {Cout,Sum} with expected. Report zero mismatches only after simulation is actually executed.

8. Sample test vectors

A	B	Cin	Expected {Cout,Sum}
0000	0000	0	0_0000
0011	0101	0	0_1000
1111	0001	0	1_0000
1111	1111	1	1_1111

9. Tools and deliverables

Suggested tools: Icarus Verilog or ModelSim for functional simulation, GTKWave for waveform viewing, and Yosys for synthesis. Submit source code, testbench, waveform screenshot, synthesis summary, and a comparison discussion.

10. Expected outcome and limitations

Expected: logically correct four-bit addition, subject to passing verification. Carry delay increases with each cascaded stage; no fabricated timing, power or silicon-area measurements are reported.

11. Conclusion

The project provides a realistic foundation for RTL design, exhaustive verification, and digital arithmetic architecture comparison.

12. References and further reading

M. M. Mano and M. D. Ciletti, Digital Design; N. H. E. Weste and D. Harris, CMOS VLSI Design. Check editions and page references used in the final submission.